

Домашнее задание 3

Q1. Уникальные рейсы и сортировка

```
SELECT DISTINCT flight_no FROM flights ORDER BY flight_no;
```

```
SELECT flight_no FROM flights GROUP BY flight_no ORDER BY flight_no;
```

1.1. DISTINCT vs GROUP BY

- Итоговый набор данных одинаков: для одного столбца без агрегатов DISTINCT и GROUP BY делают ровно одно и то же — убирают повторяющиеся значения flight_no, NULL в обоих случаях считается одним значением.
- В PostgreSQL результат этих запросов всегда одинаков, а план обычно совпадает (HashAggregate). Если вход отсортирован, в EXPLAIN могут различаться названия узлов: Unique у DISTINCT и GroupAggregate у GROUP BY, но работа они выполняют эквивалентную. Оптимизатор считает эти формы запроса равнозначными, поэтому выбор между ними на результат не влияет.
- Реальная разница: GROUP BY позволяет считать агрегаты по группам (COUNT(*), MIN(...)), а DISTINCT только убирает дубликаты.

1.2. ORDER BY без ASC и регистр символов

- ASC — сортировка по умолчанию по стандарту SQL, поэтому ORDER BY flight_no ≡ ORDER BY flight_no ASC.
- 'PG0001' и 'pg0001' — это разные строки (SQL различает регистр при сравнении текста), поэтому DISTINCT не считает их дубликатами и в выборке останутся обе.
- При сортировке по умолчанию (ORDER BY flight_no) они остаются разными значениями, и то, какая из двух встанет раньше (заглавная или строчная версия) и будут ли они стоять рядом, зависит от настроек сравнения текста в конкретной базе, а не задаётся самим SQL-запросом. Поэтому для предсказуемого порядка на практике данные обычно приводят к единому регистру.

Q2. Аэропорты и выручка за билеты

```
SELECT f.flight_id, f.flight_no,  
       COALESCE(SUM(tf.amount), 0) AS revenue,  
       COUNT(DISTINCT t.passenger_id) AS passengers  
FROM flights f  
LEFT JOIN ticket_flights tf ON tf.flight_id = f.flight_id  
LEFT JOIN tickets t ON t.ticket_no = tf.ticket_no  
GROUP BY f.flight_id, f.flight_no;
```

2.1. Один пассажир — два билета на один рейс

- SUM(amount) суммирует строки ticket_flights, то есть билеты: два билета одного пассажира — это две строки, и обе суммы корректно входят в выручку (платит каждый билет отдельно).
- А вот COUNT(*) в этом запросе посчитал бы строки соединения, то есть билеты, а не людей — пассажир с двумя билетами был бы учтён дважды. Поэтому число пассажиров считают через COUNT(DISTINCT t.passenger_id).
- При LEFT JOIN для рейса без билетов COUNT(*) вернул бы 1 (строка рейса с NULL-ами), а не 0. Это ещё одна причина считать пассажиров через COUNT(DISTINCT t.passenger_id)

2.2. Рейс без единого проданного билета

- Так как мы используем LEFT JOIN, рейс не пропадёт из отчёта, даже если на него не продали ни одного билета. Для такого рейса в столбцах билетов просто будут пустоты (NULL).
- COALESCE(SUM(...), 0) заменяет NULL на 0: для рейса без билетов SUM по строкам с NULL возвращает NULL.

- COUNT(DISTINCT ...) при подсчёте пассажиров игнорирует NULL и для такого рейса возвращает 0.
- INNER JOIN, наоборот, оставляет только пары строк с совпавшим ключом — рейс без билетов отсеивается ещё на этапе соединения и не появится в результате вовсе (не «0», а отсутствие строки).

Q3. Отменённые рейсы и города

```
SELECT DISTINCT a.city
FROM airports a
WHERE NOT EXISTS (
  SELECT 1
  FROM flights f
  JOIN airports d ON d.airport_code = f.departure_airport
  WHERE d.city = a.city AND f.status = 'Cancelled'
);
```

3.1. Аэропорт, которого нет в flights

- Основа запроса — таблица airports, а не flights, поэтому город, для которого в flights вообще нет ни одной строки, всё равно попадает в перебор.
- Если из какого-то города вообще никогда не было рейсов, то подзапрос ничего не найдёт и вернёт 0 строк. Для команды NOT EXISTS это означает: «Да, отменённых рейсов не обнаружено». В результате город честно попадает в итоговый список.
- Если бы строился запрос через airports JOIN flights, города без единого рейса потерялись бы при соединении — это была бы ошибка.

3.2. NULL и ловушка NOT IN

- Проблема с NOT IN в том, что он пытается поочерёдно сравнить город с каждым элементом из списка. Если в списке отменённых рейсов случайно окажется неизвестное значение (NULL), база данных не сможет понять, равен ему город или нет, и выдаст «неизвестный результат» вместо «правды». По правилам SQL, если условие не даёт чёткую «правду», строка отбрасывается. В итоге из-за одного NULL весь запрос может вернуть пустой список.
- NOT EXISTS не сравнивает внешнее значение со списком, а лишь проверяет, найдётся ли хотя бы одна подходящая строка — реагирует только на TRUE/FALSE, поэтому NULL внутри подзапроса не создаёт проблему.
- В подзапросе стоит WHERE status = 'Cancelled': строки с status IS NULL условие не проходят и отбрасываются. Ловушка NOT IN сработала бы только при NULL в самом выбираемом столбце подзапроса (например, departure_airport). NOT EXISTS такой проблемы не имеет, потому что проверяет лишь наличие строки, а не сравнивает значение со списком.

Q4. Места на борту и дубликаты

```
SELECT t.passenger_id, MIN(t.passenger_name) AS passenger_name
FROM boardings b
JOIN tickets t ON t.ticket_no = b.ticket_no
WHERE b.flight_id = 1 -- id нужного рейса
GROUP BY t.passenger_id;
```

4.1. Два посадочных талона у одного пассажира

- Дубликаты устраняются группировкой по passenger_id: два посадочных талона одного пассажира дают одну строку. Группировка только по passenger_id надёжнее, чем DISTINCT по паре (passenger_id, passenger_name): если имя в разных билетах записано по-разному, пара дала бы две строки.
- Для базы данных здесь нет разницы в скорости или способе обработки: и DISTINCT, и GROUP BY работают одинаково. Выбирать между ними нужно по смыслу:

- Используй **DISTINCT**, если нужно просто убрать дубликаты и получить чистый список.
- Используй **GROUP BY**, если помимо удаления дубликатов требуется что-то посчитать или объединить (например, узнать количество посадочных талонов или скомпоновать список мест).

4.2. Сдан билет, но есть посадочный талон

- Запрос выше соединяет только boardings и tickets по ticket_no — таблица ticket_flights в нём не участвует, поэтому такую аномалию запрос не заметит: пассажир с сданным билетом, но существующим талоном, попадёт в результат.
- Если добавить ещё INNER JOIN ticket_flights (например, чтобы посчитать выручку), пассажир из результата пропадёт — для него больше нет пары в ticket_flights.

Q5. DDL и блокировки на живой таблице

5.1. ALTER TABLE ADD COLUMN при открытой транзакции с SELECT

- ALTER TABLE ... ADD COLUMN — DDL-команда, берёт блокировку ACCESS EXCLUSIVE, несовместимую ни с одним другим режимом доступа к таблице.
- Транзакция BEGIN; SELECT COUNT(*) FROM flights; держит свою блокировку на таблице до COMMIT/ROLLBACK — в PostgreSQL блокировки снимаются только в конце транзакции. Пока эта транзакция открыта, ACCESS EXCLUSIVE получить нельзя, и ALTER TABLE встаёт в очередь и ждёт.
- Сам SELECT продолжает выполняться нормально, но пока ALTER ждёт своей очереди, все новые запросы к flights (даже обычный SELECT) встают в очередь за ним — таблица фактически недоступна для всех до завершения открытой транзакции.

5.2. ADD COLUMN ... DEFAULT false в PostgreSQL 11+

- Начиная с PostgreSQL 11, ALTER TABLE ... ADD COLUMN ... DEFAULT const с не-volatile значением выполняется мгновенно: меняются только метаданные в системном каталоге, существующие строки физически не переписываются. Блокировка ACCESS EXCLUSIVE всё равно берётся, но кратковременно.
- Более позднее ALTER TABLE ... ALTER COLUMN SET DEFAULT true меняет значение по умолчанию только для будущих вставок, которые не указывают колонку явно; уже существующие строки не трогаются.
- Параллельные INSERT: вставки, зафиксированные до применения новой команды, получают старое значение по умолчанию (false); вставки, выполненные после — новое (true). Сама SET DEFAULT снова кратковременно берёт ACCESS EXCLUSIVE и ждёт своей очереди — та же логика блокировок, что в 5.1.

5.3. CREATE INDEX на живой таблице

- Обычный CREATE INDEX (без CONCURRENTLY) берёт блокировку SHARE. Она несовместима с ROW EXCLUSIVE, которую берут INSERT/UPDATE/DELETE, поэтому вся запись в таблицу ждёт до конца построения индекса; SELECT (ACCESS SHARE) с SHARE совместим и продолжает работать.
- Чтобы не блокировать запись, используют CREATE INDEX CONCURRENTLY — она не мешает ни чтению, ни записи.
- Компромисс: CONCURRENTLY делает два прохода по таблице и ждёт завершения ранее начатых транзакций, из-за чего строится заметно дольше и сильнее грузит CPU/IO; не может выполняться внутри транзакции; при сбое может оставить в базе INVALID-индекс, который нужно удалить и пересоздать. Поэтому на небольших таблицах или в окно обслуживания обычный CREATE INDEX проще.

Сводка по блокировкам:

Команда	Блокировка	SELECT	INSERT/UPDATE/DELETE
ALTER TABLE ... ADD COLUMN	ACCESS EXCLUSIVE	ждёт	ждёт

CREATE INDEX	SHARE	работает	ждёт
CREATE INDEX CONCURRENTLY	SHARE UPDATE EXCLUSIVE	работает	работает

Q6. Агрегация и NULL в вычислениях

6.1. AVG(actual_departure - scheduled_departure)

**SELECT AVG(actual_departure - scheduled_departure) AS avg_delay
FROM flights;**

- Если actual_departure = NULL (рейс отменён или ещё не вылетел), разность тоже NULL. Агрегатные функции (в том числе AVG), кроме COUNT(), игнорируют NULL: AVG считается как сумма не-NULL значений, делённая на количество не-NULL значений.
- Значит рейсы без фактического времени вылета просто не участвуют в расчёте — они не превращаются в «нулевую задержку» и не искажают среднее в ту или иную сторону. Итог — средняя задержка по фактически вылетевшим рейсам, а не по всем.

6.2. COUNT(aircraft_code) vs COUNT(*)

- COUNT(aircraft_code) считает только строки, где значение не NULL, COUNT(*) — все строки без исключения.
- Результаты совпадают, если в столбце нет NULL, и различаются, если есть рейсы с aircraft_code IS NULL (например, самолёт ещё не назначен) — разница равна количеству таких строк.
- Связь с ограничением NOT NULL: если на aircraft_code стоит NOT NULL, NULL в этом столбце в принципе невозможны, и оба COUNT всегда совпадают. Без такого ограничения на уровне схемы совпадение результатов ничем не гарантировано.